



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Mrs. Mamatha
Student Name	Nehasree Padamati
Roll Number	2520030174
Branch	CSE
Course Outcome	CO1 – Trees & Balanced Search Structures

Case Study Topic: Document Tracker – Document Index Management using BST and AVL Tree

Work Done Summary:

In this case study, we implemented a Binary Search Tree (BST) and an AVL Tree to manage document records in the Document Tracker system. Each document is stored as a node keyed by its unique DocumentID. The BST supports fast lookup, insertion, and deletion of documents. Because continuous additions and removals can skew a plain BST, we upgraded to an AVL Tree that self-balances after every operation, guaranteeing $O(\log n)$ worst-case performance. In-order traversal of the AVL Tree yields documents sorted by ID, making it easy to generate organized document listings and track records efficiently.

Implementation Details:

1. Modelled each document as a node: DocumentID (int), documentName (String), owner (String), status (String).
2. Implemented BST insert, search, and delete operations.
3. Extended to AVL Tree with balance-factor tracking and LL / RR / LR / RL rotations.
4. Inserted 10 document records and verified rotations at each step.
5. Performed in-order traversal to produce a sorted document listing.
6. Benchmarked AVL Tree vs plain BST for 1,000 random insertions; AVL remained balanced and delivered consistent search performance.

Result: Screenshots / Output

```
=== AVL Tree Execution ===
```

```
Inserted:
```

```
1001 1045 1078 1102 1134
```

```
1167 1203 1240 1289 1310
```

```
RR Rotation at 1001
```

```
RR Rotation at 1078
```

```
RR Rotation at 1167
```

```
In-Order Traversal:
```

```
1001 1045 1078 1102 1134
```

```
1167 1203 1240 1289 1310
```

```
AVL Tree Height = 4
```

```
Execution Successful
```

The AVL Tree maintained balanced structure throughout all insertions. Sample sorted destination IDs after 10 insertions: 1001, 1045, 1078, 1102, 1134, 1167, 1203, 1240, 1289, 1310.

Time Complexity:

- BST Insert / Search / Delete (avg): $O(\log n)$
- AVL Insert / Delete (worst): $O(\log n)$
- In-order Traversal: $O(n)$

AVL Tree outperforms plain BST for skewed / sequential DestinationID inputs.

GitHub Link: https://github.com/nehasree2507-hash/DSA2-CaseStudies_Sem3

Student Signature	Faculty Signature